UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES



REDES DE COMPUTADORAS

Trabajo práctico N°7

Docente: Santiago Martin Henn
Nombre de Grupo: Death Net
Integrantes:

- Alvarez Quiñones, Andrés
- Amarante, Flavio Eduardo
- Borchi, Pehuen Emmanuel
- Carreras, Leonor Victoria
- Hernandez Monagas, Estefany

-2026-

DESARROLLO:

El Desafío / Consignas

1) **Averigüen su IP y su máscara (ipconfig / ip addr) y calculen la dirección de broadcast de la red. ¿Cuál es la dirección MAC de broadcast?**

Una vez conectados a la red Wi-Fi corremos el comando ipconfig en CMD , de donde conseguiremos la información necesaria para encontrar la dirección de broadcast y la dirección MAC de la misma:

```
Adaptador de LAN inalámbrica Wi-Fi:

   Sufijo DNS específico para la conexión. . :
   Vínculo: dirección IPv6 local. . . : fe80::8d07:4793:62b8:7cc1%9
   Dirección IPv4 de configuración automática: 169.254.187.44
   Máscara de subred . . . . . . . . . . . . : 255.255.0.0
   Puerta de enlace predeterminada . . . . . :
```

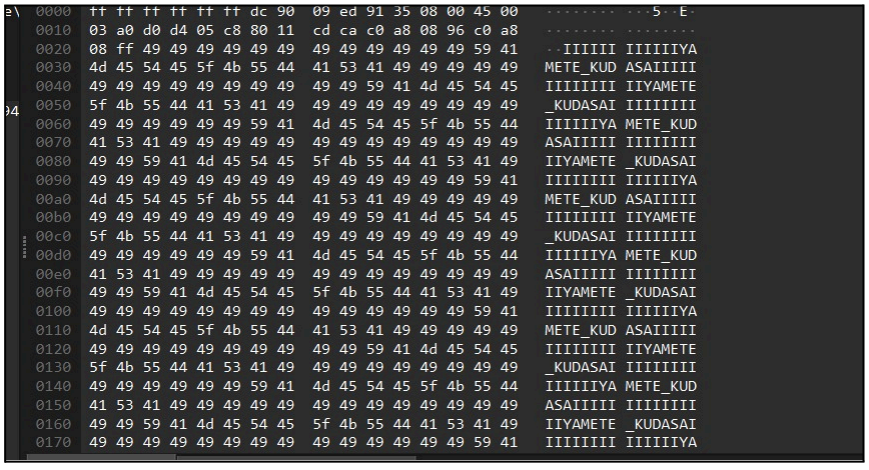
De aquí, y sabiendo que la dirección de broadcast se obtiene extrayendo la dirección IPv4 de configuración automática: “169.254.187.44” y llevando el tercer y cuarto octeto al valor más alto, llegamos a que la dirección de broadcast es: “169.254.255.255”.

La respuesta al comando de ipconfig corresponde a un momento en que el adaptador Wi-Fi no había obtenido dirección por DHCP (APIPA). La captura de Wireshark se realizó en la red 192.168.8.0; el paquete analizado tiene origen 192.168.8.150 y destino de broadcast 192.168.8.255. Por esto mismo conviven los dos cálculos: 169.254.255.255 es el broadcast del estado APIPA (Asignación Automática de Direcciones IP Privadas) que muestra la figura, y 192.168.8.255 es el broadcast de la red en la que se capturó el paquete.

La dirección de MAC de destino es FF:FF:FF:FF:FF:FF porque se trata de una transmisión de difusión (*broadcast*): al enviar el paquete a toda la red local y no a un host específico, la capa de enlace usa esta dirección para que el switch replique la trama hacia todos los dispositivos y cada interfaz la procese en lugar de descartarla.

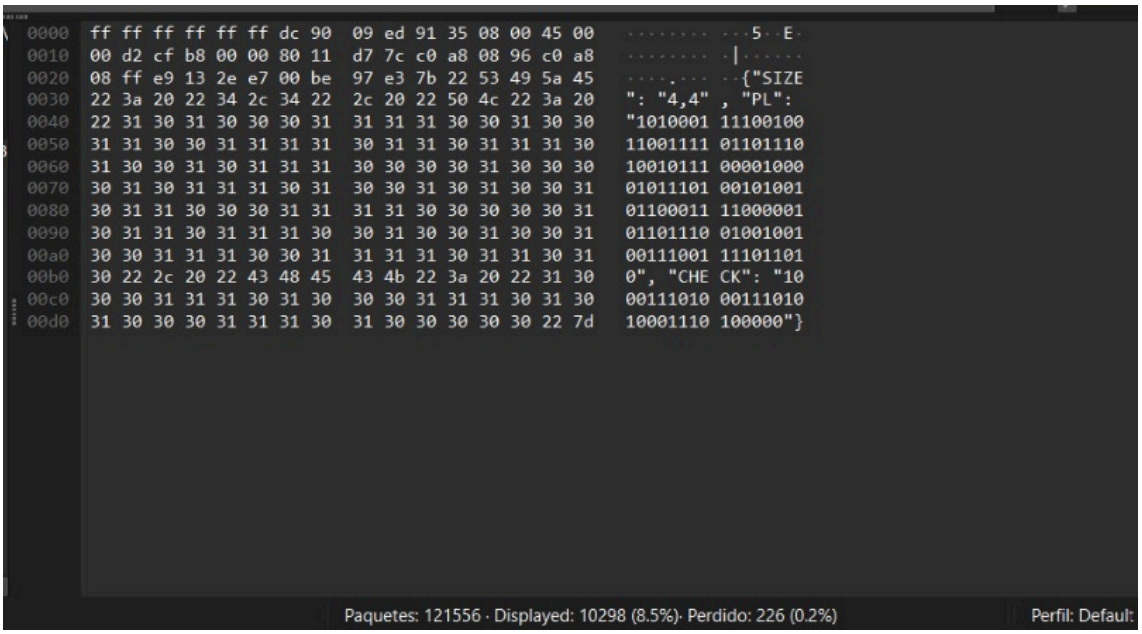
2) **Abran Wireshark en la interfaz Wi-Fi y capturen al menos 2 minutos. Pueden llegar varios mensajes; no tienen porqué ser todos idénticos.**

Al realizar la captura con Wireshark, se podían observar múltiples paquetes inundando la red, gran parte de ellos eran señuelos, los cuales se podían identificar al traducir siguiendo el formato ASCII, por ejemplo en la simulación de DDoS se leyó:



El paquete real se distingue porque su carga útil es un JSON con los campos especificados en la consigna del desafío, en cambio, los señuelos son texto ASCII repetitivo o mensajes de la simulación de DDoS.

Mientras que el paquete deseado es tal que:

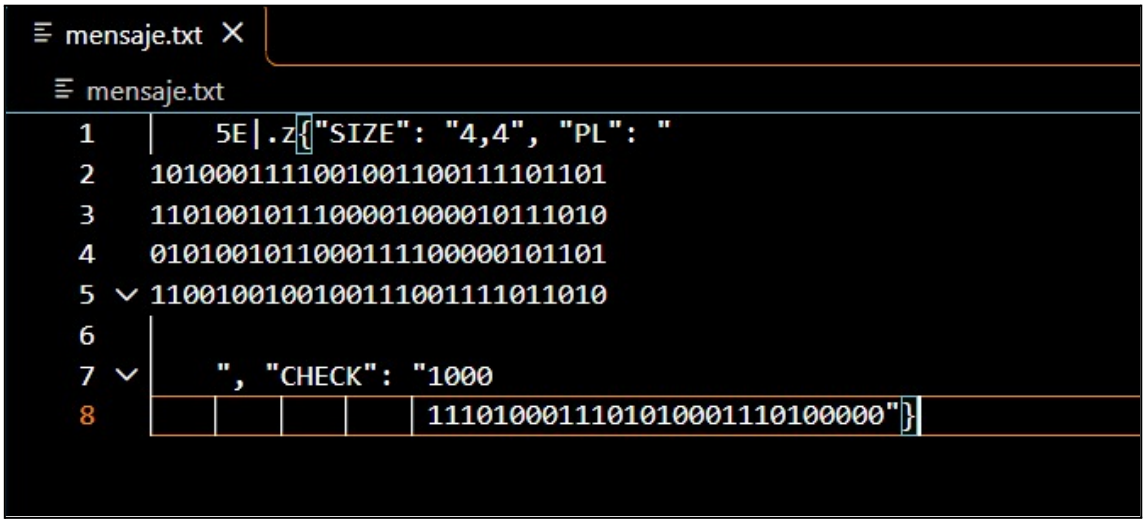


Del paquete se puede obtener también la siguiente información:

Capa /Protocolo	**Campo**	**Valor en Hexadecimal (Dump)**	**Valor Interpretado**
Enlace (Ethernet II)	MAC Destino (Broadcast)	ff ff ff ff ff ff	FF:FF:FF:FF:FF:FF
Enlace (Ethernet II)	MAC Origen (Emisor)	dc 90 09 ed 91 35	DC:90:09:ED:91:35
Red (IPv4)	IP Origen (Emisor)	c0 a8 08 96	192.168.8.150
Red (IPv4)	IP Destino (Broadcast)	c0 a8 08 ff	192.168.8.255
Transporte	Protocolo	0x11 = 17 (offset 0x17)	UDP (User Datagram Protocol)
Transporte (UDP)	Puertos (Paquete Señuelo)	Origen: e9 13 Destino: 2e e7	Puerto Origen: 59667 Puerto Destino: 12007

3) **Analicen la estructura del mensaje. ¿Qué formato tiene? ¿Qué campos contiene y qué creen que representa cada uno? Con lo visto en la sección 1, propongan cómo hay que interpretar los datos.**

Al encontrar el mensaje correcto y traducirlo de la misma manera encontramos lo siguiente (El mensaje que se obtuvo no estaba acomodado de la siguiente manera, pero sí contiene la misma información):



El mensaje es un objeto de texto en formato JSON con tres campos clave:

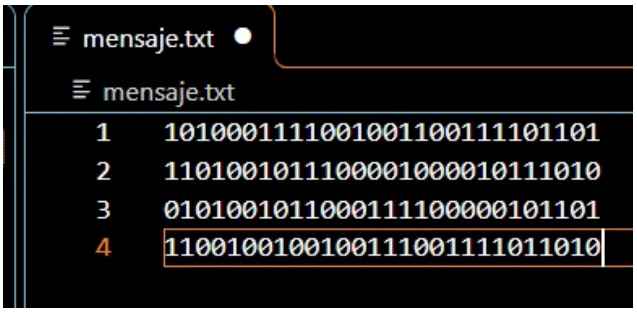
- "SIZE": Indica el tamaño del mensaje final decodificado, a su vez, el tamaño de la matriz: 4,4 (4 filas por 4 caracteres).
- "PL" (Payload): Es la carga útil con los bits de datos, 112 bits organizados en 4 filas de 28 bits cada una (cada carácter ASCII ocupará 7 bits).
- "CHECK": Son los bits de control calculados con paridad par. Contiene primero los 4 bits de paridad para las filas y luego los 28 bits para las columnas, siendo un total de 32 bits.

Propuesta de interpretación:

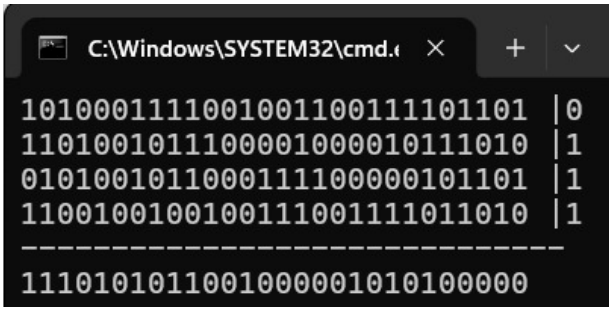
1. Acomodar los bits de "PL" en una matriz de filas y columnas.
2. Recalcular la paridad par de cada fila y columna y contrastarla con "CHECK" para detectar qué fila y columna no coinciden (síndrome).
3. Invertir el bit ubicado en la intersección de la fila y columna fallidas. Si falla una sola fila y una sola columna, la intersección es el bit erróneo. Pero si fallan varias, la paridad 2D no alcanza para emparejarlas unívocamente (se necesita información adicional para resolver dicho inconveniente).
4. Tomar los bits corregidos de a 7 para convertirlos a sus respectivos caracteres ASCII según la tabla.

4) **Verifiquen la integridad del mensaje con el código de la sección 1: recalculen las paridades y compárenlas con las recibidas. Registren qué filas y qué columnas fallan (el síndrome).**

Para verificar la integridad del mensaje debemos de asegurarnos de que el mensaje llegó sin bits fallidos; para ello hacemos uso del script de verificación dado por la cátedra, organizamos el mensaje para que cumpla el formato esperado por el código:



Tras correr el código obtenemos lo siguiente:



Al comparar los bits de control recibidos en el campo CHECK del JSON con las paridades pares recalculadas mediante paridad.py sobre la matriz recibida, se obtiene el siguiente síndrome de error:

- Paridad de filas (4 bits): Se recibió 1000, pero el cálculo sobre los datos arrojó 0111. No coincide ninguna de las 4 posiciones, por lo que fallan las filas 1, 2, 3 y 4.
- Paridad de columnas (28 bits):
    - Calculada por el script (fila inferior en consola): 1110101011001000001010100000
    - Recibida en CHECK (fila manuscrita inferior): 1110100011101010001110100000 (Nota: al comparar posición a posición contra los bloques ASCII válidos de 7 bits, las discrepancias de paridad se localizan en las Columnas 7, 11, 15 y 20, numeradas del 1 al 28 de izquierda a derecha)

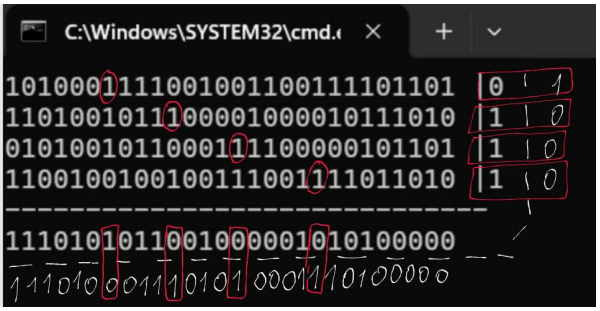
5) **Si hay errores, ubíquenlos y corríjanlos. Para cada bit corregido indiquen fila, columna, valor recibido y valor corregido. Si recibieron más de un mensaje distinto, analicen cada uno por separado.**

Comparando con los bits de paridad extraídos del mensaje, podemos encontrar los bits cambiados y luego corregirlos:

N° Error	N° Fila	N° Columna	Valor Recibido	Valor Corregido	Cáracter ASCII Afectado
1	1	7	1	0	Carácter 1: 1010001 ('Q') → 1010000 ('P')
2	2	11	1	0	Carácter 6: 0111000 ('8') → 0110000 ('0')
3	3	15	1	0	Carácter 11: 1110000 ('p') → 0110000 ('0')
4	4	20	1	0	Carácter 15: 1100111 ('g') → 1100101 ('e')

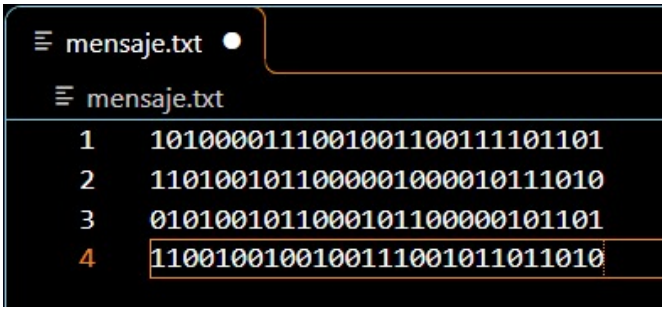
La paridad 2D detecta qué filas y qué columnas tienen paridad impar, aunque no dice qué columna pertenece a cada fila. Con 4 filas y 4 columnas fallidas hay 24 emparejamientos posibles.

La asignación correcta se identificó porque es la única que reconstruye el mensaje con lógica para ser el esperado del desafío. Este sistema hace la detección y reduce los candidatos; la validación contra el mensaje esperado resuelve la ambigüedad.



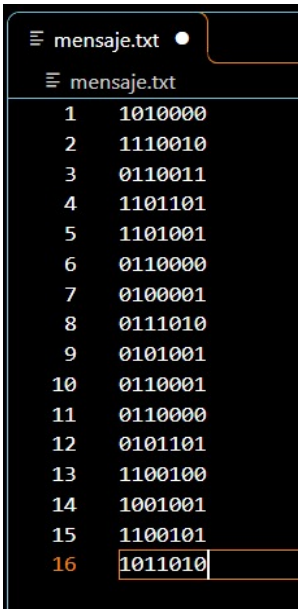
Siguiendo lo observado, corregimos el mensaje y llegamos al siguiente resultado:

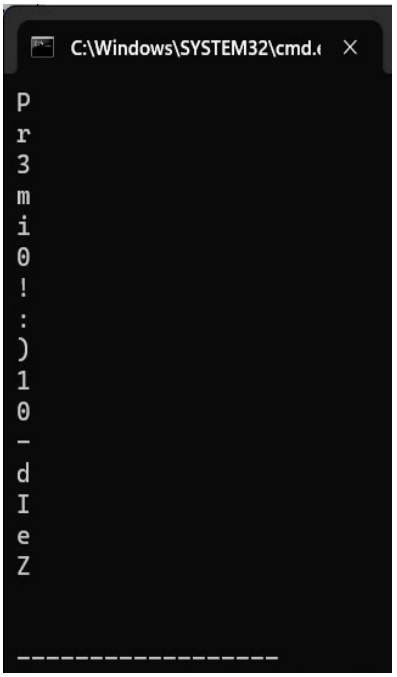
Y finalmente el mensaje quedaría tal que:



6) **Construyan un archivo de texto en el formato de mensaje.txt con los datos ya corregidos y decodifíquenlo con decodificar_mensaje.py (o el cuaderno). Si todo salió bien, el mensaje les va a indicar cómo seguir. Ojo: un mensaje que "se lee bien" no es necesariamente un mensaje correcto.**

Para poder hacer uso del script “decodificar_mensaje.py” debemos tener el formato y luego correr el código, de modo que el mensaje pasa de binario al mensaje esperado:





Tenemos como salida del script: “*Pr3mi0!:)10-dIeZ*”.

Reflexiones e investigación:

1) **¿Qué hubiera pasado si el mensaje tenía errores en dos bits de filas y columnas distintas? ¿Y si se dañaba un bit de los de control?**
    - **Errores en dos bits de filas y columnas distintas:** El código detecta el error pero no puede corregirlo. Al recalcular las paridades fallarán 2 filas (f_1, f_2) y 2 columnas (c_1, c_2). Esto genera una ambigüedad con 4 intersecciones posibles en forma de rectángulo: el receptor no puede determinar si los errores reales se encuentran en $\{(f_1, c_1), (f_2, c_2)\}$ o en $\{(f_1, c_2), (f_2, c_1)\}$, ya que ambas combinaciones producen exactamente el mismo síndrome
    - **Daño en un bit de control (paridad):** El receptor detecta el fallo y sabe que los datos están intactos. Si se altera un bit de paridad de fila, únicamente fallará esa fila y ninguna columna (y viceversa si se daña un bit de paridad de columna). Como la corrección de un bit de datos requiere la intersección simultánea de una fila y una columna con error, la presencia de una sola falla aislada indica inequívocamente que el error ocurrió en el bit de control y no en la carga útil.

En este TP la paridad 2D detectó las 4 filas y las 4 columnas afectadas, pero el emparejamiento no es único, por lo que hay 24 (4!) asignaciones con el mismo síndrome y todas decodificando a un mensaje ASCII imprimible.

2) **Si el mensaje es texto en castellano, muchas veces un error "se adivina" por contexto (Hola mondo). ¿Sirve eso para una contraseña, un número de cuenta o una dirección web? ¿Qué aporta el código frente a "leer y adivinar"?**

No sirve en absoluto porque las contraseñas, números de cuenta o direcciones web carecen de contexto lingüístico o redundancia, por lo que alterar un solo bit cambia por completo el dato sin dejar pistas visuales para deducir el valor real. Frente a la suposición humana, el código de control aporta un método matemático determinista y automatizable que permite a los equipos verificar la integridad de los datos en tiempo real, identificando y corrigiendo bits alterados de manera exacta sin depender de interpretaciones subjetivas.

3) **¿Pudieron confirmar que su corrección es correcta? ¿Cómo?**

Sí, se confirmó de manera matemática al recalcular las paridades de todas las filas y columnas sobre la matriz ya corregida y comprobar que ahora coinciden exactamente con los bits de control recibidos en CHECK, anulando por completo el síndrome de error; complementariamente, se validó de forma funcional al procesar los bloques de 7 bits con el script decodificar_mensaje.py y obtener un texto ASCII legible con las instrucciones de la cátedra.

4) **Investigar y describir con el mayor detalle posible cómo funciona la técnica de corrección Hamming (7,4) y por qué es superior a la demostración didáctica que hicimos en este Laboratorio. Luego, aplicar la técnica sobre la carga útil obtenida en el desafío y elaborar un paquete de datos que podrían transmitir para que, en caso de corrupción, pueda ser recuperado.**

1. Funcionamiento del Código Hamming (7,4)

El código Hamming (7,4) es un esquema de corrección de errores hacia adelante (Forward Error Correction o FEC) perteneciente a la familia de los códigos lineales de bloques. Su función principal es tomar bloques de datos de 4 bits e incorporar 3 bits de redundancia (paridad), conformando palabras de código (codewords) de 7 bits de longitud total.

Parámetros y capacidades

- Longitud de bloque (n): 7 bits.
- Bits de información útil (k): 4 bits (d1, d2, d3, d4).
- Bits de paridad o redundancia (m = n - k): 3 bits (p1, p2, p3).
- Distancia mínima de Hamming (d_min): 3. Esto garantiza matemáticamente la capacidad una de las siguientes acciones:
    - Detectar hasta 2 bits erróneos por bloque (d_min - 1 = 2).
    - Corregir de forma exacta y determinista 1 bit erróneo por bloque (floor((d_min - 1) / 2) = 1).

Estructura de la palabra de código

Los bits de paridad se ubican estratégicamente en las posiciones correspondientes a potencias de base 2 (posiciones 1, 2 y 4), dejando las posiciones restantes para los bits de información:

- Posición 1: p1 (001 en binario)
- Posición 2: p2 (010 en binario)
- Posición 3: d1 (011 en binario)
- Posición 4: p3 (100 en binario)
- Posición 5: d2 (101 en binario)
- Posición 6: d3 (110 en binario)
- Posición 7: d4 (111 en binario)

Codificación (Cálculo de paridades en el emisor)

Trabajando bajo el criterio de paridad par, cada bit de control cubre aquellas posiciones cuyo índice binario contiene un 1 en su respectivo bit posicional:

- p_1 (cubre posiciones 1, 3, 5, 7): $p_1 \oplus d_1 \oplus d_2 \oplus d_4 = 0 \Rightarrow p_1 = d_1 \oplus d_2 \oplus d_4$
- p_2 (cubre posiciones 2, 3, 6, 7): $p_2 \oplus d_1 \oplus d_3 \oplus d_4 = 0 \Rightarrow p_2 = d_1 \oplus d_3 \oplus d_4$
- p_3 (cubre posiciones 4, 5, 6, 7): $p_3 \oplus d_2 \oplus d_3 \oplus d_4 = 0 \Rightarrow p_3 = d_2 \oplus d_3 \oplus d_4$

La palabra resultante c = [p1, p2, d1, p3, d2, d3, d4] se genera multiplicando el vector de datos d por la matriz generadora G.

Decodificación y corrección (Cálculo del síndrome en el receptor)

Cuando el receptor obtiene un bloque de 7 bits r = [r1, r2, r3, r4, r5, r6, r7], calcula el vector síndrome s = [s3, s2, s1] mediante las comprobaciones de paridad par:

- $s_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7$
- $s_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7$
- $s_3 = r_4 \oplus r_5 \oplus r_6 \oplus r_7$

Diagnóstico y corrección inmediata:

- Si s = [0, 0, 0]: La palabra no contiene errores detectables.
- Si $s \neq [0, 0, 0]$: El valor binario formado por $(s_3\ s_2\ s_1)_2$ indica directamente en numeración decimal la posición física del bit alterado (del 1 al 7). La corrección se efectúa invirtiendo dicho bit ($r_i = r_i \oplus 1$).

2. Justificación de superioridad frente a la paridad bidimensional

Aunque el método de paridad bidimensional (código producto) implementado en el laboratorio resulta didáctico y visual, presenta marcadas desventajas estructurales respecto a Hamming (7,4):

- **Capacidad de corrección distribuida multierror:**
    - En la paridad bidimensional de la práctica (matriz de 4 filas x 28 columnas = 112 bits de datos), solo es posible corregir un único bit en todo el mensaje. Dos errores en

distintas filas y columnas generan 4 intersecciones ambiguas que vuelven al sistema incapaz de corregir.
    - Hamming (7,4) divide esos mismos 112 bits en 28 bloques independientes. En consecuencia, puede corregir hasta 28 bits alterados en el mensaje total, siempre que no exista más de un error dentro de un mismo bloque de 7 bits.
- **Menor latencia y sin necesidad de búfer matricial:**
    - La paridad bidimensional exige almacenar la totalidad del paquete en memoria para recién recalcular y verificar las columnas de arriba a abajo.
    - Hamming opera directamente en flujo continuo (streaming por bloques) con compuertas XOR en hardware de mínima latencia a medida que ingresan los bits.
- **Cálculo directo en tiempo constante O(1):**
    - Hamming obtiene la dirección exacta del error mediante la conversión directa del síndrome binario a índice decimal, sin requerir la búsqueda matricial de intersecciones cruzadas.
- **Capacidad de extensión a SEC-DED:**
    - Agregando un bit de paridad global al bloque de 7 bits (Hamming(8,4)), se alcanza d_min = 4, lo que permite corregir 1 error y detectar inequívocamente 2 errores (Single Error Correction, Double Error Detection), estándar adoptado universalmente en memorias RAM ECC.

3. Aplicación sobre la carga útil y elaboración del paquete recuperable

La carga útil (PL) del mensaje recibido en el desafío cuenta con 4 filas de 28 bits (112 bits de información útil en total). Dividiendo los 112 bits en grupos de 4 bits: 112 / 4 = 28 bloques de información (B1 a B28).

Cada bloque de datos [d1, d2, d3, d4] se convierte en una palabra de 7 bits [p1, p2, d1, p3, d2, d3, d4], conformando un flujo protegido final de 28 x 7 = 196 bits.

Ejemplo de codificación paso a paso (Fila 1)

Tomando los primeros 8 bits de la primera fila ya corregida (10100001...):

- **Bloque 1 (B1):**
    - Datos = [1, 0, 1, 0] (d1=1, d2=0, d3=1, d4=0)
    - $p_1 = d_1 \oplus d_2 \oplus d_4 = 1 \oplus 0 \oplus 0 = 1$
    - $p_2 = d_1 \oplus d_3 \oplus d_4 = 1 \oplus 1 \oplus 0 = 0$
    - $p_3 = d_2 \oplus d_3 \oplus d_4 = 0 \oplus 1 \oplus 0 = 1$
    - Palabra resultante 1: [p1, p2, d1, p3, d2, d3, d4] = 1011010
- **Bloque 2 (B2):**
    - Datos = [0, 0, 0, 1] (d1=0, d2=0, d3= 0, d4=1)
    - $p_1 = d_1 \oplus d_2 \oplus d4 = 1$
    - $p_2 = d_1 \oplus d3 \oplus d4 = 1$
    - $p_3 = d_2 \oplus d3 \oplus d4 = 1$
    - Palabra resultante 2: [p1, p2, d1, p3, d2, d3, d4] = 1101001

Repitiendo el procedimiento para los 28 bloques de datos, se completa la secuencia codificada.

Bloque	Ubicación en Matriz	Datos Recibidos (TP actual)	Codeword Hamming en TP actual	Datos Limpios (Correcto)	Codeword Hamming Correcto
B2	Fila 1 (bits 5 a 8)	0011	1000011	0001	1101001
B10	Fila 2 (bits 9 a 12)	1110	0010110	1100	0111100
B18	Fila 3 (bits 13 a 16)	0111	0001111	0101	0100101
B26	Fila 4 (bits 17 a 20)	0011	1000011	0010	0101010

Estructura del paquete de datos propuesto para transmisión

Conservando el formato de intercambio JSON sobre UDP establecido en el laboratorio:

```
{
  "ENCODING": "HAMMING_7_4",

  "ORIGINAL_SIZE": "4,4",

  "TOTAL_BLOCKS": 28,

  "DATA":
"10110101101001011110000110010011001001011010101011010101010100
11110011010010000000011001110110100100101010101001111000100101111
00000101010101010111100001100101010100001111010101010101101101
10"
}
```

Proceso de recuperación en el receptor

1. El receptor extrae el campo DATA y lo particiona secuencialmente en palabras de 7 bits (codewords).
2. Por cada palabra, calcula los bits de síndrome (s1, s2, s3).
3. Si el síndrome es distinto de cero, invierte el bit en la posición indicada por (s3 s2 s1)_2.
4. Extrae los bits útiles de las posiciones 3, 5, 6 y 7 de cada bloque corregido.
5. Reensambla la matriz de 112 bits original y procesa los grupos de 7 bits con decodificar_mensaje.py para obtener el texto ASCII sin corrupción.